

■ Design Distributed Key-Value Store

System Design Interview | Distributed Infrastructure

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	GET(key), PUT(key, value), DELETE(key); TTL/expiry; Versioning; Batch operations
Non-Functional	Low latency (p99 <10ms); High availability (99.999%); Horizontal scalability; Fault tolerant; Tunable consistency
Scale	Petabyte-scale data; Millions of keys; Handle node failures gracefully; Consistent hashing for distribution

■ Core Design Components

- **Consistent Hashing:** distributes keys across nodes; adding/removing nodes rebalances only K/N keys (vs K keys with mod hashing)
- **Virtual Nodes (vnodes):** each physical node owns multiple positions on hash ring; better load distribution; Amazon Dynamo uses 150-200 vnodes/node
- **Replication:** replicate each key to N nodes (e.g., N=3); coordinator node writes to all N replicas; read from R replicas
- **Quorum:** $W+R > N$ ensures strong consistency; W=2, R=2, N=3 is common; trade latency for consistency

■ Conflict Resolution & Versioning

- **Vector Clocks:** Attach (node, counter) pairs to each value version; detect causality and conflicts; merge or surface conflict to client
- **Last-Write-Wins (LWW):** Use timestamp to pick winner; simple but loses data; DynamoDB default; acceptable for many use cases
- **CRDTs:** Conflict-free Replicated Data Types: mathematically guaranteed to merge without conflicts; good for counters, sets
- **Sloppy Quorum:** During partition, accept writes on reachable nodes; hinted handoff stores data for failed node owner; repair on recovery

■ Storage Engine (LSM Tree)

- Writes go to in-memory MemTable (sorted) + WAL (for crash recovery); fast $O(\log N)$ in-memory write
- When MemTable full → flush to SSTable (immutable sorted file on disk); SSTables periodically compacted
- Bloom filter per SSTable: $O(1)$ check if key exists before disk read; eliminates unnecessary I/O
- Read path: check MemTable → bloom filter → SSTable (newest to oldest); compaction reduces SSTable count
- vs B-Tree: LSM better write throughput; B-Tree better read performance; LSM used by Cassandra, RocksDB

■■ Consistency Models

Strong Consistency	Eventual Consistency
✓ $W=N, R=1$ or $W=1, R=N$	✓ $W=1, R=1$ (async replication)
✓ All reads see latest write	✓ Reads may be stale
✓ Higher latency	✓ Lower latency, higher availability
✓ Zookeeper, etcd	✓ DynamoDB, Cassandra default

■ Anti-Entropy & Failure Detection

- Gossip Protocol: each node periodically shares its state with K random neighbors; failure info propagates in $O(\log N)$ rounds
- Merkle Trees: hash tree of key ranges; compare trees between replicas to find inconsistencies efficiently; sync only diverged ranges
- Read Repair: during quorum read, if replicas disagree → write latest version back to stale replicas asynchronously
- Hinted Handoff: if target node is down, coordinator stores write with hint; when target recovers, hint is delivered

■ Real-World Examples

- Amazon DynamoDB: managed, sloppy quorum, LWW conflicts, adaptive capacity, global tables for multi-region
- Apache Cassandra: masterless, tunable consistency (ONE/QUORUM/ALL), LWT for conditional writes, wide-column store
- Redis: single-node (fast), Redis Cluster for distribution, Sentinel for HA; primarily in-memory with persistence options
- etcd/Zookeeper: strong consistency (Raft/ZAB consensus), small data, used for coordination/config not general KV